

Pertemuan 2: Class, Object & Class Diagram

Teguh Sutanto, M.Kom.

PERTEMUAN 2 • 150 MENIT

Agenda hari ini

Kita mulai dari analogi sederhana sampai bisa gambar diagram dan coding aggregation. Semua hands-on di Java.

TARGET AKHIR:
Mampu bedakan Class vs Object dan buat Class Diagram dengan relasi yang benar.

01 **Review Pertemuan 1**
Prosedural vs OOP

02 **Analogi Class vs Object**
Cetakan kue → kue jadi

03 **Struktur Class di Java**
Attribute + method

04 **Membuat Object**
Keyword new & heap

05 **Class Diagram UML**
3 kotak + visibilitas

06 **Relasi Diagram**
Association, Aggregation, Composition

07 **Live Coding Demo**
Peminjaman punya Buku

08 **Praktikum Kelompok**
Inventory elektronik

Review: apa itu OOP?

Prosedural = resep masakan
Langkah demi langkah, fungsi ke fungsi

OOP = objek yang hidup
Menciptakan objek yang saling berinteraksi

J

Java = Pure OOP. Semua kode wajib di dalam Class. File name = Class name.

Encapsulation

Menjaga data biar aman, akses via getter setter

Inheritance

Warisan sifat dengan extends, reuse code

Polymorphism

Satu bentuk banyak wujud, override

Abstraction

Fokus ke yang penting, sembunyikan kompleksitas

ANALOGI INTI PBO

Cetakan kue



C Class = Cetakan kue (Blueprint)

Definisi, struktur, bukan barang jadi. Hanya ada 1 cetakan.

O Object = Kue yang jadi (Instance)

Hasil cetakan, punya data unik rasa, topping, ukuran.

∞ 1 cetakan → 100 kue beda rasa

Efisiensi: tulis Class sekali, buat Object sepuasnya.

CONTOH REAL KAMPUS

Class Mahasiswa → Object:
Budi (NIM 001, IF) , Ani (NIM 002, SI)
new Mahasiswa() = adonan baru

Struktur class di Java

Blueprint yang punya data dan perilaku — semua di dalam satu file yang namanya sama.

Mahasiswa.java

```
public class Mahasiswa {

// Attribute = data
  String nama;
  String nim;


// Method = perilaku

void perkenalan() {
    System.out.println(
"Halo " + nama);
  }
}
```

1 **File name = Class name**

Mahasiswa.java wajib berisi public class Mahasiswa. Huruf besar di awal.

2 **Attribute = data**

Seperti kolom di Excel. Contoh: nama, nim, alamat.

3 **Method = perilaku**

Apa yang bisa dilakukan object itu. Contoh: perkenalan(), belajar().

! Di Java, tidak ada kode di luar class. Semua harus di dalam.

Membuat object di Java

Main.java

```
Mahasiswa mhs1 = new Mahasiswa();
mhs1.nama =
"Budi";
mhs1.nim =
"123";
mhs1.perkenalan();

Mahasiswa mhs2 = new Mahasiswa();
mhs2.nama =
"Ani";

// 1 cetakan → 2 objek berbeda
```

new — **Keyword new = memanggil constructor**

Constructor mengalokasikan memori, mengembalikan reference ke object. Tanpa new, hanya deklarasi reference null.

HEAP — **Object disimpan di heap memory**

mhs1 & mhs2 hidup di heap, variable reference ada di stack. Beda alamat, beda data meskipun class sama.



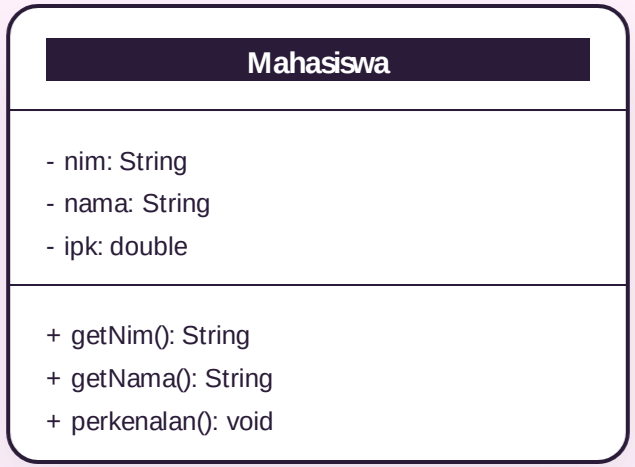
UML • BLUEPRINT CODE

Class diagram UML

1. **Kotak 3 bagian:**
   Nama Class (Bold) → Attribute → Method

2. **Visibilitas standar UML:**
   + public, - private, # protected, ~ package

3. **Tools wajib:**
   StarUML (gratis), Visual Paradigm, draw.io untuk cek cepat



! Nama file .java harus sama persis dengan nama Class. Case-sensitive.

VISIBILITAS

- private	dalam class saja
+ public	dari mana saja
# protected	class + child
~ package	satu package

StarUML | Visual Paradigm | draw.io

Contoh penulisan

```
- nim: String
+ nama: String {readOnly}
+ hitungSKS(): int
```

Visibilitas & encapsulation

Visibilitas mengatur siapa yang boleh akses. Private bukan pelit — ini kunci keamanan dan validasi data.



– private — hanya dalam Class itu

Atribut sensitif seperti nim, saldo. Tidak bisa diakses langsung dari luar.

+ public — bisa di mana saja

Method umum seperti perkenalan(), getNim(), setNim()

protected — Class + Child

Diwariskan ke subclass, dipakai untuk inheritance nanti

? Kenapa harus private?

Untuk validasi & keamanan. Akses data hanya lewat getter setter, jadi kita bisa cek sebelum disimpan.

```
private String nim;
public String getNim(){ return nim; }
public void setNim(String n){ if(n.length()==3) this.nim=n; }
```

Relasi dalam class diagram

Relasi = bagaimana class saling terhubung. Beda simbol, beda makna hidup-mati. Fokus utama P2 adalah Aggregation (◇) yang jadi kunci tugas inventory.

— Association

GARIS LURUS

Hubungan biasa, saling kenal. Tidak memiliki.

Mahasiswa — MataKuliah
mengambil

◇ Aggregation

PUTIH

Punya-a, tapi bagian bisa hidup sendiri.

Perpustakaan ◇– Buku
Buku tetap ada tanpa perpustakaan

◆ Composition

HITAM

Bagian-dari, hidup-mati ikut induk.

Rumah ◆– Kamar
Kamar tidak ada tanpa Rumah

◁ Generalization

INHERITANCE

Segitiga kosong. Parent ke child.

Pegawai ◁– Dosen
Dosen extends Pegawai

-- Realization

INTERFACE

Garis putus-putus. Class implements contract.

Class ..|▶ Interface
Mis: Barang implements Printable

INGAT P2

Aggregation = punya-a, bukan bagian-dari

Live coding: aggregation

JAVA • HAS-A RELATION

Peminjaman.java

```
public class Peminjaman {

Buku buku; // Aggregation ◇

Anggota anggota;
  String tglPinjam;


public Peminjaman(Buku b, Anggota a, String tgl){

this.buku = b;

this.anggota = a;

this.tglPinjam = tgl;
  }

void info(){
    System.out.println(anggota.nama
      +
" pinjam " + buku.judul);
  }
}
```

CLASS DIAGRAM • AGGREGATION



Buku & Anggota tetap hidup tanpa Peminjaman → aggregation (berlian putih ◇)

OUTPUT CONSOLE

```
> Budi pinjam PBO Java tgl 21-09-2026
```

✓ **Key:** Object di-inject lewat constructor, bukan di-new di dalam. Lifecycle terpisah.

PRAKTIKUM KELOMPOK • 8 KELOMPOK

Sistem inventory

Studi kasus: Toko elektronik

Kelola stok masuk dari supplier, catat barang & jumlah

1 Buat Class Diagram di StarUML

Wajib ada minimal 1 relasi Aggregation ◇

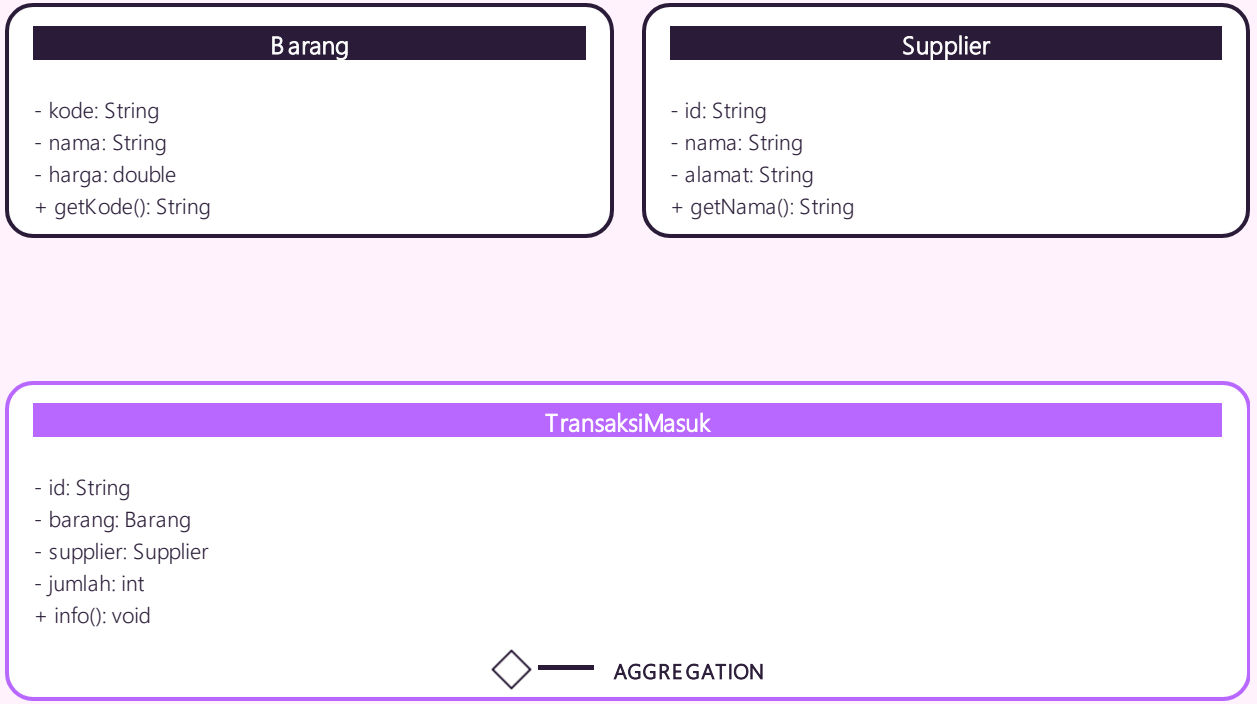
2 Implementasi 3 Class di Java

Barang, Supplier, TransaksiMasuk dengan atribut private

3 Presentasi terbaik → Inovatif 10%

2 kelompok terpilh demo di depan kelas

★ Inovatif 10% untuk diagram paling rapi + logika aggregation benar



RULE TransaksiMasuk ◇— Barang : Barang bisa hidup sendiri tanpa transaksi. Beda dengan Composition ◆

Tugas & penilaian P2

Upload rapi = nilai rapi. Ikuti format ZIP dan lampirkan 3 bukti: diagram, kode, screenshot jalan.

FORMAT UPLOAD

NIM_Nama_P2.zip

1. **JPG** Class Diagram (export StarUML, min 1080p)
2. **3 file .java** : Barang, Supplier, TransaksiMasuk
3. **PNG** screenshot output console / GUI running

DEADLINE — **1 minggu di Moodle**

Late = -10% per hari. Upload di Assignment P2, bukan forum.

RUBRIK 100%

Diagram	40%
Kode Aggregation	30%
Jalan	20%
Rapi	10%

KONTRIBUSI NILAI

13% dari CLO052

Tugas	1%
Inovatif 10%	presentasi terbaik
Praktikum	2%

✓ Checklist sebelum upload

Diagram ada aggregation ◇ , kode ada this.buku, screenshot console ada output peminjaman

MOODLE P2

PERTEMUAN 2 SELESAI • RANGKUMAN & NEXT

Class = Blueprint of Object, Object = Instance of Class

C

Class itu cetakan

Definisi struktur, bukan data jadi. 1 file, 1 class.

O

Object itu instance

Hasil new, hidup di heap, punya alamat beda.

◇

Aggregation = punya-a

Peminjaman punya Buku, tapi Buku bisa hidup sendiri.

Q

Kuis cepat: Apa beda Aggregation vs Composition?

Tulis 1 baris kode Aggregation! Contoh: Peminjaman(buku, anggota)

NEXT P3 • JANGAN LUPA

Attribute, Method, Modifier, Overloading, static. Install StarUML malam ini, coba gambar diagram Mahasiswa-MataKuliah.

JAVA 17 • INTELLIJ • STARUML • kampus.db — Sampai jumpa di lab!